My Main.go

```
package main

import (
"context"
"fmt"
"log"
"net/http"
"os"
"os/signal"
"syscall"
"time"

"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/shared/database"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/shared/kafka"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/shared/redis"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/user-management/config"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/user-management/middleware"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-service/handlers"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-service/models"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-service/repositories"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-service/routes"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-service/services"
"github.com/Automobile-Association-of-Kenya/aa-core-system/pkg/auth"
"github.com/gin-gonic/gin"
"github.com/joho/godotenv"
"gorm.io/gorm"
)

// @title AA Core System - Valuation Service API
// @version 1.0
// @description Vehicle valuation service for AA Core System
// @contact.name API Support
// @contact.email support@aa.co.ke
// @host localhost:8084
```

```
// @BasePath /api/v1
// @securityDefinitions.apikey BearerAuth
// @in header
// @name Authorization

func main() {
// Load environment variables
if err := godotenv.Load(); err != nil {
log.Println("No .env file found, using system environment variables")
}

// Load configuration
cfg, err := config.LoadConfig("./configs")
if err != nil {
log.Fatalf("Failed to load config: %v", err)
}

// Set Gin mode
gin.SetMode(cfg.Server.Mode)

// Initialize database
dbConfig := database.Config{
Host: cfg.Database.Host,
Port: cfg.Database.Port,
User: cfg.Database.User,
Password: cfg.Database.Password,
DBName: "aa_valuation_service", // Separate database for valuation service
SSLMode: cfg.Database.SSLMode,
MaxOpenConns: cfg.Database.MaxOpenConns,
MaxIdleConns: cfg.Database.MaxIdleConns,
ConnMaxLifetime: cfg.Database.ConnMaxLifetime,
}

db, err := database.NewPostgresConnection(dbConfig)
if err != nil {
log.Fatalf("Failed to connect to database: %v", err)
}

// Run migrations
if err := runMigrations(db); err != nil {
log.Fatalf("Failed to run migrations: %v", err)
}

// Initialize Redis
```

```
redisConfig := redis.Config{
Host: cfg.Redis.Host,
Port: cfg.Redis.Port,
Password: cfg.Redis.Password,
DB: 3, // Different Redis DB for valuation service
PoolSize: cfg.Redis.PoolSize,
}

redisClient, err := redis.NewRedisClient(redisConfig)
if err != nil {
log.Fatalf("Failed to connect to Redis: %v", err)
}

// Initialize Kafka
kafkaConfig := kafka.Config{
Brokers: cfg.Kafka.Brokers,
Topics: map[string]string{
"user_events": "user-events",
"valuation_events": "valuation-events",
"partner_events": "partner-events",
"agent_events": "agent-events",
"valuation_type_events": "valuation-type-events",
"branch_events": "branch-events",
"vehicle_insurance_events": "vehicle-insurance-events",
"valuer_events": "valuer-events",
"valuation_assignment_events": "valuation-assignment-events",
"notification_events": "notification-events",
},
}

kafkaProducer := kafka.NewProducer(kafkaConfig)
defer kafkaProducer.Close()

// Initialize JWT manager
jwtManager := auth.NewJWTManager(
cfg.JWT.Secret,
cfg.JWT.AccessTokenDuration,
cfg.JWT.RefreshTokenDuration,
)

// Initialize repositories
customerRepo := repositories.NewCustomerRepository(db)
partnerRepo := repositories.NewPartnerRepository(db)
agentRepo := repositories.NewAgentRepository(db)
```

```
valuationTypeRepo := repositories.NewValuationTypeRepository(db)
branchRepo := repositories.NewBranchRepository(db)
vehicleInsuranceRepo := repositories.NewVehicleInsuranceRepository(db)
valuerRepo := repositories.NewValuerRepository(db)
// Add other repositories as you implement them

// Initialize services
customerService := services.NewCustomerService(customerRepo, kafkaProducer)
partnerService := services.NewPartnerService(partnerRepo, kafkaProducer)
agentService := services.NewAgentService(agentRepo, partnerRepo, kafkaProducer)
valuationTypeService := services.NewValuationTypeService(valuationTypeRepo,
kafkaProducer)
branchService := services.NewBranchService(branchRepo, kafkaProducer)
vehicleInsuranceService :=
services.NewVehicleInsuranceService(vehicleInsuranceRepo, customerRepo,
kafkaProducer)
valuerService := services.NewValuerService(valuerRepo, kafkaProducer)
// Add other services as you implement them

// Initialize default data
if err := initializeDefaultData(valuationTypeService, branchService); err != nil {
log.Printf("Warning: Failed to initialize default data: %v", err)
}

// Initialize middleware
authMiddleware := middleware.NewAuthMiddleware(jwtManager, nil, redisClient)

// Initialize handlers
customerHandler := handlers.NewCustomerHandler(customerService)
partnerHandler := handlers.NewPartnerHandler(partnerService)
agentHandler := handlers.NewAgentHandler(agentService)
valuationTypeHandler := handlers.NewValuationTypeHandler(valuationTypeService)
branchHandler := handlers.NewBranchHandler(branchService)
vehicleInsuranceHandler :=
handlers.NewVehicleInsuranceHandler(vehicleInsuranceService)
valuerHandler := handlers.NewValuerHandler(valuerService)
// Add other handlers as you implement them

// Initialize Gin router
router := gin.New()
router.Use(gin.Recovery())

// Health check endpoint (no auth required)
router.GET("/health", func(c *gin.Context) {
```

```
c.JSON(200, gin.H{"status": "healthy", "service": "valuation-service"})
})

// Setup routes
routes.SetupRoutes(router, customerHandler, partnerHandler, agentHandler,
valuationTypeHandler, branchHandler, vehicleInsuranceHandler, valuerHandler,
authMiddleware)
// Add other routes as you implement them
// routes.RegisterValuationRequestRoutes(router, valuationRequestHandler,
authMiddleware)

// Start Kafka consumers in background
go startKafkaConsumers(kafkaConfig, customerService, partnerService, agentService,
valuationTypeService, branchService, vehicleInsuranceService, valuerService)

// Start scheduled tasks
go startScheduledTasks(vehicleInsuranceService)

// Create HTTP server
srv := &http.Server{
Addr: ":8084", // Different port for valuation service
Handler: router,
ReadTimeout: cfg.Server.ReadTimeout,
WriteTimeout: cfg.Server.WriteTimeout,
}

// Start server in a goroutine
go func() {
log.Printf("Starting valuation service on port 8084")
if err := srv.ListenAndServe(); err != nil && err != http.ErrServerClosed {
log.Fatalf("Failed to start server: %v", err)
}
}()

// Wait for interrupt signal to gracefully shutdown the server
quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit
log.Println("Shutting down valuation service...")

ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()

if err := srv.Shutdown(ctx); err != nil {
```

```
log.Fatalf("Server forced to shutdown: %v", err)
}

log.Println("Valuation service shutdown complete")
}

// runMigrations handles database migrations for valuation service
func runMigrations(db *gorm.DB) error {
log.Println("Running valuation service migrations...")

err := db.AutoMigrate(
&models.Customer{},
&models.Partners{},
&models.Agents{},
&models.ValuationTypes{},
&models.Branches{},
&models.VehicleInsurances{},
&models.Valuer{},
&models.ValuationRequests{},
&models.ValuationInvoices{},
&models.ValuationPayments{},
)

if err != nil {
return err
}

log.Println("Valuation service migrations completed successfully")
return nil
}

// initializeDefaultData initializes default valuation types and branches
func initializeDefaultData(valuationTypeService *services.ValuationTypeService,
branchService *services.BranchService) error {
// Initialize default valuation types
if err := valuationTypeService.InitializeDefaultValuationTypes(); err != nil {
return fmt.Errorf("failed to initialize default valuation types: %v", err)
}

// Initialize default branches
if err := branchService.InitializeDefaultBranches(); err != nil {
return fmt.Errorf("failed to initialize default branches: %v", err)
}
```

```
return nil
}

// startScheduledTasks starts background scheduled tasks
func startScheduledTasks(vehicleInsuranceService *services.VehicleInsuranceService)
{
// Check for expiry notifications every 24 hours
ticker := time.NewTicker(24 * time.Hour)
defer ticker.Stop()

// Run immediately on startup
if err := vehicleInsuranceService.CheckExpiryNotifications(); err != nil {
log.Printf("Failed to check expiry notifications: %v", err)
}

for {
select {
case <-ticker.C:
if err := vehicleInsuranceService.CheckExpiryNotifications(); err != nil {
log.Printf("Failed to check expiry notifications: %v", err)
}
}
}
}

// startKafkaConsumers handles Kafka event consumption
func startKafkaConsumers(kafkaConfig kafka.Config, customerService
*services.CustomerService, partnerService *services.PartnerService, agentService
*services.AgentService, valuationTypeService *services.ValuationTypeService,
branchService *services.BranchService, vehicleInsuranceService
*services.VehicleInsuranceService, valuerService *services.ValuerService) {
consumer := kafka.NewConsumer(kafkaConfig, "valuation-service")

// Handle user events from user-management service
go func() {
err := consumer.ConsumeEvents("user_events", func(event kafka.Event) error {
switch event.Type {
case "user.registered":
return customerService.HandleCustomerCreated(event)
case "user.email_verified":
return customerService.HandleCustomerVerified(event)
default:
log.Printf("Unhandled user event type: %s", event.Type)
}
```

```
return nil
})
if err != nil {
log.Printf("Failed to consume user events: %v", err)
}
}()

// Handle partner events (if needed for cross-service communication)
go func() {
err := consumer.ConsumeEvents("partner_events", func(event kafka.Event) error {
switch event.Type {
case "partner.created":
log.Printf("Partner created: %v", event.Data)
case "partner.updated":
log.Printf("Partner updated: %v", event.Data)
case "partner.deleted":
log.Printf("Partner deleted: %v", event.Data)
default:
log.Printf("Unhandled partner event type: %s", event.Type)
}
return nil
})
if err != nil {
log.Printf("Failed to consume partner events: %v", err)
}
}()

// Handle agent events (if needed for cross-service communication)
go func() {
err := consumer.ConsumeEvents("agent_events", func(event kafka.Event) error {
switch event.Type {
case "agent.created":
log.Printf("Agent created: %v", event.Data)
case "agent.updated":
log.Printf("Agent updated: %v", event.Data)
case "agent.deleted":
log.Printf("Agent deleted: %v", event.Data)
default:
log.Printf("Unhandled agent event type: %s", event.Type)
}
return nil
})
if err != nil {
log.Printf("Failed to consume agent events: %v", err)
```

```
}
}()

// Handle valuation type events (if needed for cross-service communication)
go func() {
err := consumer.ConsumeEvents("valuation_type_events", func(event kafka.Event)
error {
switch event.Type {
case "valuation_type.created":
log.Printf("Valuation type created: %v", event.Data)
case "valuation_type.updated":
log.Printf("Valuation type updated: %v", event.Data)
case "valuation_type.deleted":
log.Printf("Valuation type deleted: %v", event.Data)
default:
log.Printf("Unhandled valuation type event type: %s", event.Type)
}
return nil
})
if err != nil {
log.Printf("Failed to consume valuation type events: %v", err)
}
}()

// Handle branch events (if needed for cross-service communication)
go func() {
err := consumer.ConsumeEvents("branch_events", func(event kafka.Event) error {
switch event.Type {
case "branch.created":
log.Printf("Branch created: %v", event.Data)
case "branch.updated":
log.Printf("Branch updated: %v", event.Data)
case "branch.deleted":
log.Printf("Branch deleted: %v", event.Data)
default:
log.Printf("Unhandled branch event type: %s", event.Type)
}
return nil
})
if err != nil {
log.Printf("Failed to consume branch events: %v", err)
}
}()
```

```
// Handle vehicle insurance events (if needed for cross-service communication)
go func() {
err := consumer.ConsumeEvents("vehicle_insurance_events", func(event kafka.Event)
error {
switch event.Type {
case "vehicle_insurance.created":
log.Printf("Vehicle insurance created: %v", event.Data)
case "vehicle_insurance.updated":
log.Printf("Vehicle insurance updated: %v", event.Data)
case "vehicle_insurance.deleted":
log.Printf("Vehicle insurance deleted: %v", event.Data)
default:
log.Printf("Unhandled vehicle insurance event type: %s", event.Type)
}
return nil
})
if err != nil {
log.Printf("Failed to consume vehicle insurance events: %v", err)
}
}()

// Handle valuer events (if needed for cross-service communication)
go func() {
err := consumer.ConsumeEvents("valuer_events", func(event kafka.Event) error {
switch event.Type {
case "valuer.created":
log.Printf("Valuer created: %v", event.Data)
case "valuer.updated":
log.Printf("Valuer updated: %v", event.Data)
case "valuer.deleted":
log.Printf("Valuer deleted: %v", event.Data)
default:
log.Printf("Unhandled valuer event type: %s", event.Type)
}
return nil
})
if err != nil {
log.Printf("Failed to consume valuer events: %v", err)
}
}()

// Handle valuation assignment events
go func() {
```

```
err := consumer.ConsumeEvents("valuation_assignment_events", func(event
kafka.Event) error {
switch event.Type {
case "valuer.assigned_to_request":
log.Printf("Valuer assigned to request: %v", event.Data)
default:
log.Printf("Unhandled valuation assignment event type: %s", event.Type)
}
return nil
})
if err != nil {
log.Printf("Failed to consume valuation assignment events: %v", err)
}
}()
}
```